

Introducing XML

this slide intentionally left blank

What is XML?

- eXtensible Markup Language
 - actually not a language in the strictest sense
- A way of attaching meaning to raw textual data
 - raw data interspersed with *tags* that introduce meaning
- Developed after HTML
 - both derived from SGML
 - HTML used to control display
 - XML used to convey meaning about data

```
<?xml version="1.0"?>
<message>
  <text>Hello, world</text>
</message>
```

```
<html>
  <body>
    <h1>Hello, world</h1>
  </body>
</html>
```

Markup Languages

- Used for many years
- Normally for formatting purposes
 - Unix troff
 - TeX
 - HTML
- SGML provides a standard base for markup languages

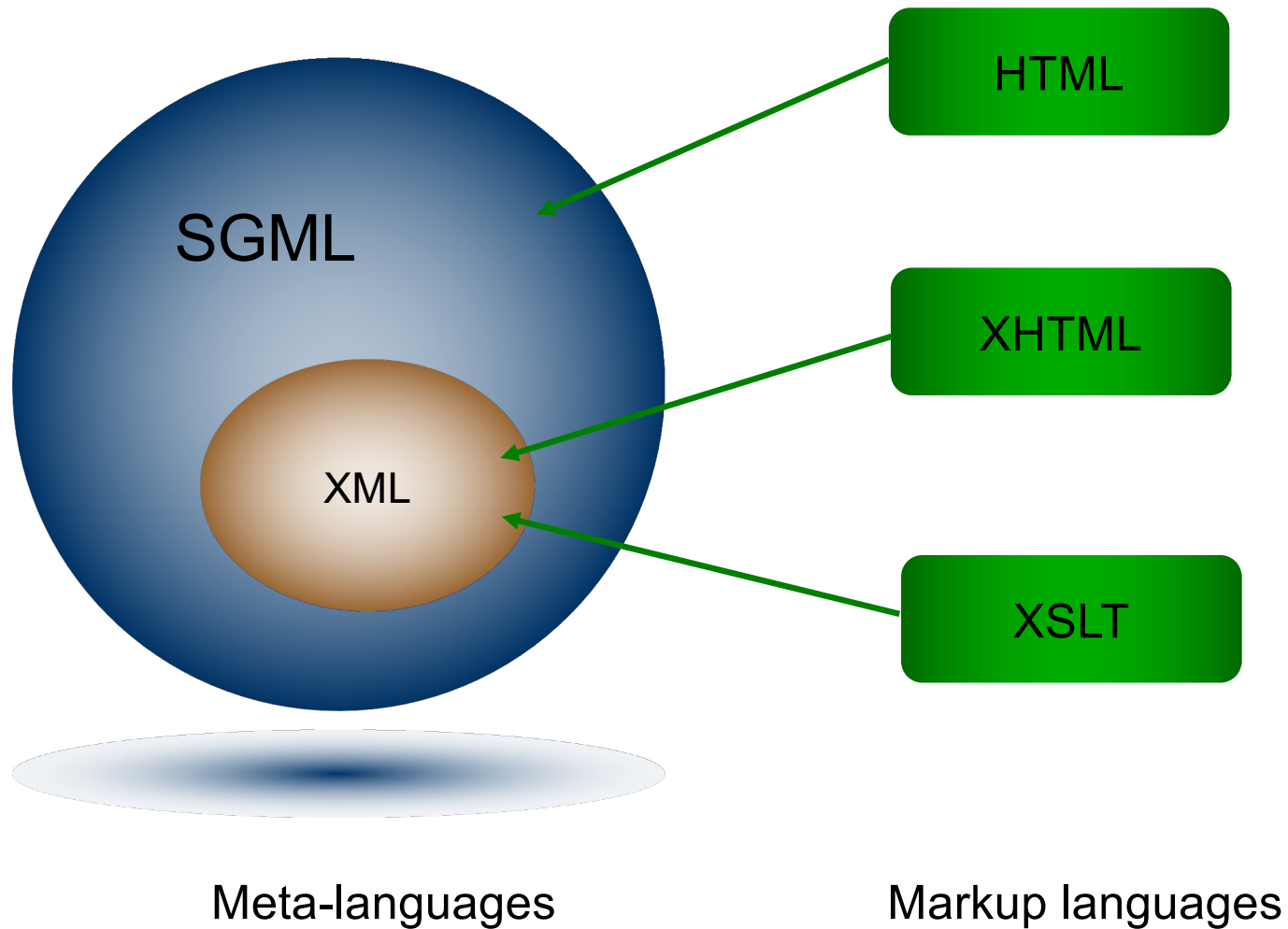
```
'\" t
.\" @(#)javac.1 1.5 98/04/20 SMI;
.\" Copyright (c) 1996 Sun Microsystems, Inc.
.\" All Rights Reserved.
'\"macro stdmacro
.TH javac 1 "8 June 1997"
.SH NAME
javac \- Java compiler
.SH SYNOPSIS
.B javac
[
.BI \-classpath " path"
] [
.BI \-d " directory"
]
```

```
\section{Getting started}
\belongs{2}
```

This lab session starts with the basics:
getting your first perl program running.

```
\begin{enumerate}
\sdu{
\question{
Get your lab environment working:
\begin{itemize}
\item Start Reflection-X
\item Open an xterm on linux
\item Open an XEmacs window on linux
\end{itemize}
}
```

SGML, HTML, XML and XHTML

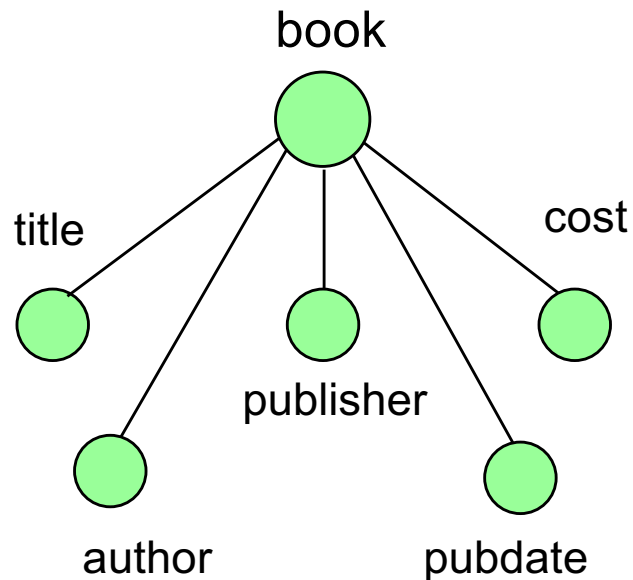


Basic XML Syntax

Document prolog

Root element
(exactly one)

```
<?xml version='1.0' ?>
<book>
  <title>Learning XML</title>
  <author>Erik T. Ray</author>
  <publisher>O'Reilly</publisher>
  <pubdate>2001</pubdate>
  <cost>34.95</cost>
</book>
```



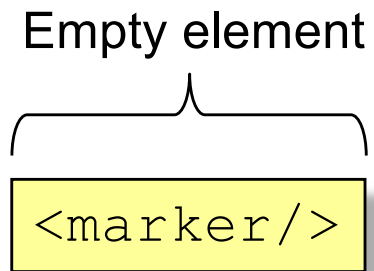
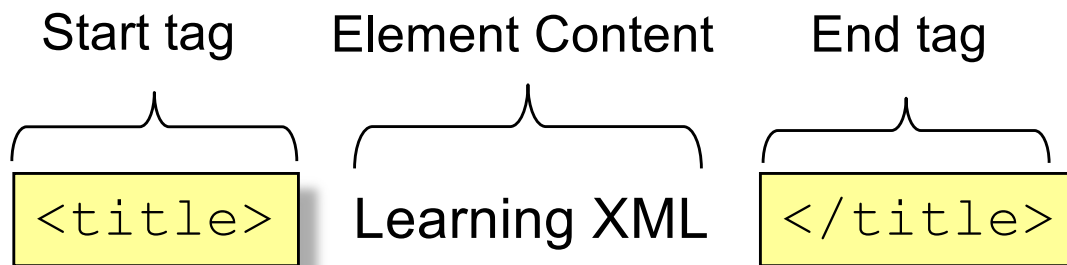
- XML implies a hierarchical document structure
 - exactly one root element

Differences from HTML

- Element tags are case sensitive
 - `<book>` and `<Book>` are different
 - beware `<author>...</Author>`
- Start and end tags are required
 - some HTML tags do not require closing tag
 - `<p>`, `<td>` etc.
- Empty element (no content) written differently
 - `<foobar/>` in XML
 - `<hr>` in HTML
- White space preserved in element content

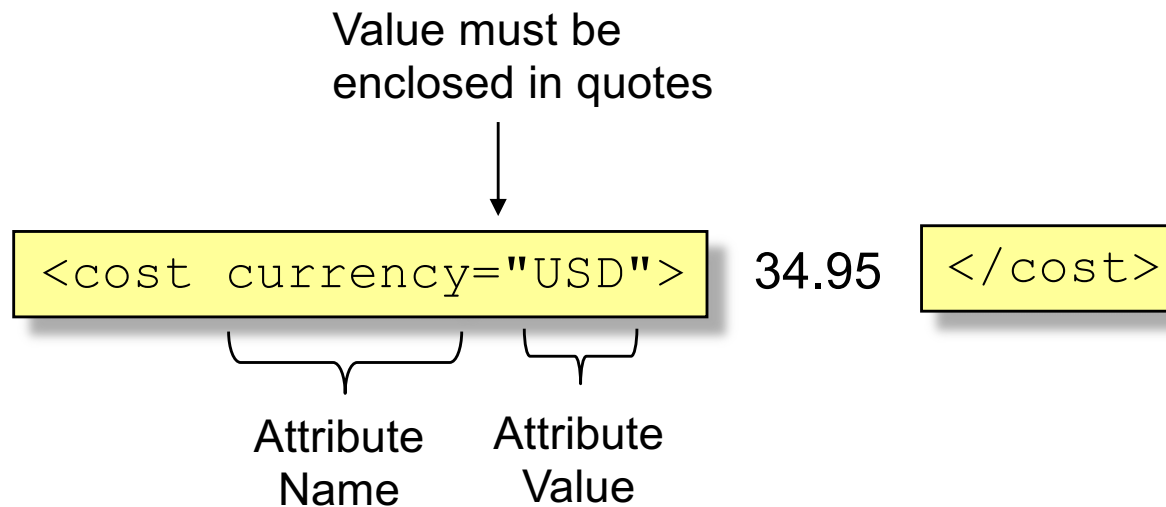
Elements

- Principal components of an XML document
- May contain text, elements or a mixture
 - or may be empty



Attributes

- Provide additional "qualifying" information to elements
- Often used with empty tags to convey information



```
<velocity magnitude="20" direction="260"/>
```

Entity References

- Not all characters can be used in textual element content
 - five significant XML characters
- Use "Entity References" instead:

For	Use
<	<
>	>
&	&
"	"
'	'

- Additional entities can be defined and used
 - more later

CDATA Sections

- Character data
 - no recognition of XML syntax
- Similar to block quoting a section of text
- Useful when incorporating certain types of content
 - program text
 - additional markup e.g. HTML

The diagram illustrates the syntax of a CDATA section. It shows the opening tag `<![CDATA[` in a yellow box, followed by the text `...stuff...`, and then the closing tag `]]>` in another yellow box. A horizontal curly brace is positioned below the text `...stuff...`, spanning from the end of the opening tag to the start of the closing tag.

No recognition of special
XML characters like <, &, " etc.

Processing Instructions and Comments

- Directives to XML processors
 - similar to "pragmas" as instructions to compilers in source code
 - beware similarity to XML declaration syntax

```
<? target text-instruction ?>
```

- Often used to embed formatting information
 - or apply stylesheet (more later)

```
<?xml-stylesheet type="text/css" href="mystyle.css"?>
```

- XML documents may also contain comments

```
<!-- This is a comment -->
```

The Document Prolog

- Header information
- Minimally contains XML declaration

```
<?xml version="1.0" standalone="yes"?>
```

Which version of XML

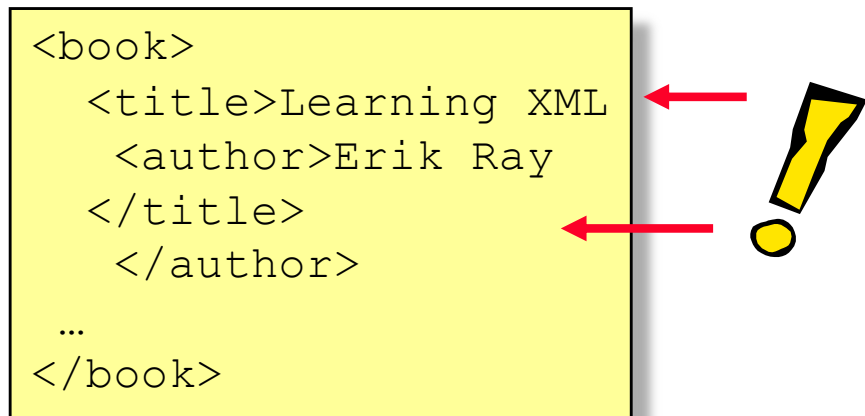
Does this document require
an external DTD for validation?

- Also specify character encoding
 - e.g. encoding="ISO-8859-1"
 - XML uses Unicode internally

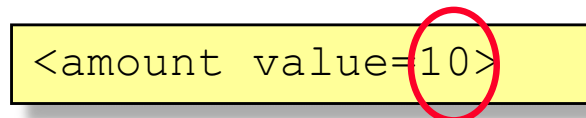
Well Formed XML

- Syntax rules more strictly applied than for HTML
- All tags must be "closed"
 - matching start and end tags
 - or empty tags
- Tags must not overlap
- Attribute values must be quoted
- Character entities must be used where necessary

```
<book>
  <title>Learning XML
  <author>Erik Ray
</title>
  </author>
...
</book>
```



```
<amount value=10>
```



Namespaces

- A namespace is a collection of element and attribute names
 - prevents collisions when elements from different sources are being used
- Element name can be qualified with namespace
 - often indicated by prefix
 - `<bk:title>`
 - `<au:title>`
- XML processor may use namespaces for other purposes
 - e.g. XSLT uses `xsl:` namespace to highlight instructions

```
<book>
  <title>Learning XML</title>
  <author>
    <title>Mr</title>
    <firstname>Erik</firstname>
    <lastname>Ray</lastname>
  </author>
  <publisher>O'Reilly</publisher>
</book>
```

Using Namespaces

- Namespace defined as an attribute of an element
 - `xmlns`
 - use URI to identify the namespace
- All child elements belong to the same namespace
 - unless they introduce a new one...

```
<book>
  <title>Learning XML</title>
  <author xmlns="http://www.gb.com/author">
    <title>Mr</title>
    <firstname>Erik</firstname>
    <lastname>Ray</lastname>
  </author>
  <publisher>O'Reilly</publisher>
</book>
```

Namespace Prefixes and Qualified Names

- Usually multiple namespaces in use at any time
- Namespace Prefix can be used to associate an element with a namespace
 - gives "qualified" element name
 - does not include child elements
 - unqualified elements belong to "default" namespace

```
<library:book  
  xmlns:library="http://www.foo.com/"  
  
  ...  
  
</library:book>
```

Unqualified elements nested inside
are *not* part of library namespace

Qualified name is
library:book

Mixing Namespace Notations

- Use xmlns attribute to set default namespace for elements
 - assumed for elements with no namespace prefix
- May be mixed with namespace prefixes and qualified names

```
<bk:book xmlns:bk="http://www.gb.com/book">  
  <bk:title>Learning XML</bk:title>
```

```
    <author xmlns="http://www.gb.com/author">  
      <title>Mr</title>  
      <firstname>Erik</firstname>  
      <lastname>Ray</lastname>  
    </author>
```

```
    <bk:publisher>O'Reilly</bk:publisher>  
</bk:book>
```

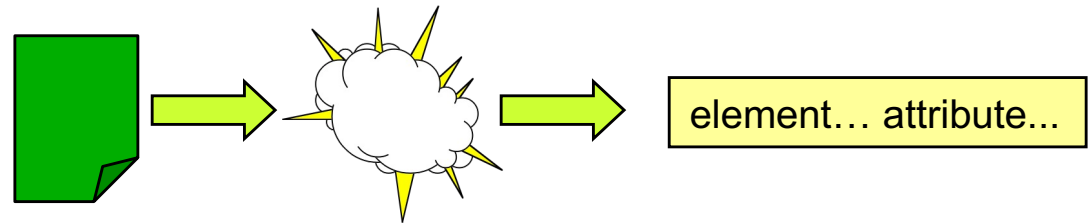
Child elements
must use
qualified name

author element
changes the
default
namespace

XML Processing Models

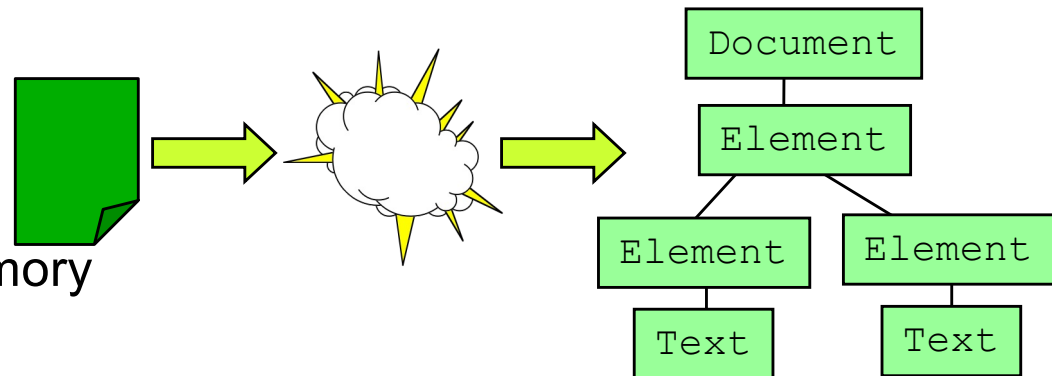
- Stream based

- XML information made available as a stream of events
- one pass through document
- usually efficient



- Tree based

- tree structure built from XML document
- all information from document kept available
- may require high memory resources



Pull vs Push Parsing

- Push Parsing
 - e.g. SAX (language independent)
 - notifications made through callbacks
 - driver program has less control over parsing
 - can lead to untidy code
- Pull Parsing
 - e.g. StAX (Java), XmlReader (.Net)
 - driver decides when to move to next component
 - more control
 - possible to skip over unwanted components

DOM

- Developed within the World Wide Web Consortium
 - "Document Object Model"
- Generic, language independent standard governing how XML documents are represented
 - bindings to different languages
 - Java, Perl, Javascript, C++, CORBA, etc.
- Present in different "levels"
 - level 1 supports basic content and navigation
 - level 2 adds options for specific content models (XHTML)
 - level 3 more sophisticated node handling, text manipulation etc.

Basic DOM Features

- Represents document internally as a tree
 - exploits inherent hierarchical structure in XML documents
- Nodes of the tree represented by Node objects
- Also specific node types
 - Element, Document, Attr, Text, etc.

```
<?xml version='1.0' ?>
<!DOCTYPE book SYSTEM "bookex.dtd">
<book>
  <title>Essential XML</title>
  <author>Don Box</author>
  <author>Aaron Skonnard</author>
  <author>John Lam</author>
  <publisher>Addison Wesley</publisher>
  <pubdate>2000</pubdate>
  <cost>23.50</cost>
</book>
```

